

TRABALHO FINAL – ESCALONADOR ROUND-ROBIN COM FEEDBACK

*Matheus Wemmer, Gabriel Soranzo Jardim, Nicolas Born, Pedro Henrique dos Santos,
Cauã Reinaldo, Ruancarlo Brum Lenzzi*

1. INTRODUÇÃO

O objetivo do trabalho consiste em montar um escalonador de processos implementando o algoritmo Round-Robin com feedback, com a função de obter maior conhecimento sobre o funcionamento de algoritmos de escalonamento de processos.

2. DESCRIÇÃO DO ALGORITMO

Round-Robin se trata de um algoritmo de escalonamento de processos que incorpora uma fatia de tempo limite (quantum), de forma que cada processo possa ser executado de forma distributiva. Ao receber seu quantum, o processo possui um intervalo para utilizar os recursos do sistema – caso não termine durante esse período, o processo regressa ao final da fila.

Nesse projeto, o algoritmo foi implementado em java.

3. PREMISSAS

3.1 – Quantum: 2 ticks

3.2 – Filas: 3 (filaAlta, filaBaixa, filaIO)

3.3 – Total de processos: 5

3.4 – Tempo I/O: 2 (mínimo) – 5 (máximo) ticks

3.5 – Tempo CPU: 4 (mínimo) – 10 (máximo) ticks

3.6 – Semente aleatória: 4576

3.7 – Critério de geração dos processos: Criados dinamicamente via objeto Random com tempos de chegada distribuídos aleatoriamente entre 1 – 10 ticks

3.8 – Regras de feedback: 1. todo processo novo entra na Fila ALTA; 2. se estourar o quantum (2 ticks), sofre preempção e cai para a Fila BAIXA; 3. retornos de I/O de DISCO vão para a Fila BAIXA; retornos de I/O de FITA ou IMPRESSORA ganham prioridade e vão para a Fila ALTA

4. ESTRUTURA DE DADOS

4.1– Representação do PCB

```
public PCB(int pid, int ppid, String prioridade, String status, int tempoChegada, int  
tempoServico,  
String tipoIO, int tempoIO, int pedidoIO) {  
    this.pid = pid;  
    this.ppid = ppid;  
    this.status = status;  
    this.prioridade = prioridade;  
    this.tempoChegada = tempoChegada;  
    this.tempoServico = tempoServico;  
    this.tempoExecucao = 0;  
    this.tempoQuantum = 0;  
    this.tipoIO = tipoIO;  
    this.tempoIO = tempoIO;
```

```

    this.pedidoIO = pedidoIO;
    this.tempoEspera = 0;
    this.tempoFinalizacao = 0;
}

```

4.2 – Representação das filas

```

ArrayList<PCB> filaAlta = new ArrayList<>();
ArrayList<PCB> filaBaixa = new ArrayList<>();
ArrayList<PCB> filaIO = new ArrayList<>();

```

5. FLUXO DE EXECUÇÃO

5.1 – Criação: os processos são gerados a partir de uma seed fixa, e então fornecidos uma conexão pipe

5.2 – Execução: um processo é garantido um intervalo de tempo onde ele pode executar (quantum)

5.3 – Preempção: ao fim do quantum, ele retorna para o final da fila baixa e o processo seguinte é executado. Retornos de I/O de disco vão para a fila baixa; retornos de I/O de fita ou impressora ganham prioridade e vão para a fila alta.

5.4 – Bloqueio: quando o processo aguarda um evento específico (exemplo: I/O)

5.5 – Finalização: quando um processo termina seu tempo de serviço, ele é finalizado

6. EXECUÇÃO

6.1 – É necessário ter o docker instalado na máquina (docker --version)

6.2 – Clone o repositório (git clone <https://github.com/matheuswemmer/escalonador-round-robin.git>) e entre na pasta do projeto: cd escalonador-round-robin

6.3 – Na raiz do projeto (onde está o Dockerfile), execute: docker build -t escalonador; esse comando irá: baixar a imagem base do Java; copiar o código fonte para o container; compilar o arquivo App.java; criar uma imagem chamada escalonador

7. SAÍDA DO SIMULADOR

7.1 – Saída

```

[config] quantum = 2 | processos = 5 | seed = 4576
[t=002] P1 criado -> fila ALTA
[t=002] P2 criado -> fila ALTA
[t=002] CPU executa P1 [fila ALTA] -> por 2 unidades
[t=004] P3 criado -> fila ALTA
[t=004] P1 sofreu preempcao -> fila BAIXA
[t=004] CPU executa P2 [fila ALTA] -> por 2 unidades
[t=005] P5 criado -> fila ALTA
[t=006] P2 sofreu preempcao -> fila BAIXA
[t=006] CPU executa P3 [fila ALTA] -> por 2 unidades
[t=008] P3 sofreu preempcao -> fila BAIXA
[t=008] CPU executa P5 [fila ALTA] -> por 2 unidades
[t=009] P4 criado -> fila ALTA
[t=010] P5 sofreu preempcao -> fila BAIXA
[t=010] CPU executa P4 [fila ALTA] -> por 2 unidades
[t=012] P4 sofreu preempcao -> fila BAIXA
[t=012] CPU executa P1 [fila BAIXA] -> por 2 unidades

```

[t=014] P1 sofreu preempcao -> fila BAIXA
 [t=014] CPU executa P2 [fila BAIXA] -> por 2 unidades
 [t=016] P2 sofreu preempcao -> fila BAIXA
 [t=016] CPU executa P3 [fila BAIXA] -> por 2 unidades
 [t=018] P3 sofreu preempcao -> fila BAIXA
 [t=018] CPU executa P5 [fila BAIXA] -> por 2 unidades
 [t=018] P5 solicitou I/O FITA por 2 unidades
 [t=019] CPU executa P4 [fila BAIXA] -> por 2 unidades
 [t=019] P4 solicitou I/O IMPRESSORA por 4 unidades
 [t=020] P5 retornou da FITA -> fila ALTA
 [t=020] CPU executa P5 [fila ALTA] -> por 2 unidades
 [t=022] P5 finalizado
 [t=022] CPU executa P1 [fila BAIXA] -> por 2 unidades
 [t=023] P4 retornou da IMPRESSORA -> fila ALTA
 [t=023] CPU executa P4 [fila ALTA] -> por 2 unidades
 [t=025] P4 finalizado
 [t=026] P1 sofreu preempcao -> fila BAIXA
 [t=026] CPU executa P2 [fila BAIXA] -> por 2 unidades
 [t=026] P2 solicitou I/O FITA por 4 unidades
 [t=027] CPU executa P3 [fila BAIXA] -> por 2 unidades
 [t=029] P3 sofreu preempcao -> fila BAIXA
 [t=029] CPU executa P1 [fila BAIXA] -> por 2 unidades
 [t=030] P2 retornou da FITA -> fila ALTA
 [t=030] CPU executa P2 [fila ALTA] -> por 2 unidades
 [t=032] P2 sofreu preempcao -> fila BAIXA
 [t=033] P1 sofreu preempcao -> fila BAIXA
 [t=033] CPU executa P3 [fila BAIXA] -> por 2 unidades
 [t=035] P3 sofreu preempcao -> fila BAIXA
 [t=035] CPU executa P2 [fila BAIXA] -> por 2 unidades
 [t=037] P2 finalizado
 [t=037] CPU executa P1 [fila BAIXA] -> por 1 unidades
 [t=038] P1 finalizado
 [t=038] CPU executa P3 [fila BAIXA] -> por 2 unidades
 [t=040] P3 finalizado
 [fim] Processos finalizados: 5/5 | tempo total: 40 | preempcoes: 13
 I/O -> DISCO: 0 | FITA: 2 | IMPRESSORA: 1
 Espera media: 17,6 | Retorno medio: 28,0
 CPU ociosa: 2 ticks (5,0%)

7.2 – Interpretação

P[X] criado -> fila ALTA: O processo X alcançou seu tempo de chegada e foi admitido na fila de prontos de maior prioridade.

CPU executa P[X] [fila ALTA/BAIXA]...: O escalonador concedeu a posse do processador ao processo X, indicando de qual fila ele foi retirado.

P[X] solicitou I/O [DISCO/FITA/IMPRESSORA] por Y unidades: O processo interceptou sua execução para aguardar uma operação de Entrada/Saída no dispositivo indicado pelo período Y.

P[X] retornou do [TIPO] -> fila [ALTA/BAIXA]: A operação de E/S foi concluída e o processo foi reescalonado na fila correspondente pelas regras de Feedback.

P[X] sofreu preempção -> fila BAIXA: O processo estourou o limite do quantum consecutivo de CPU (2 ticks) e teve seu contexto interrompido, perdendo prioridade.

P[X] finalizou: O processo consumiu todo o seu tempo de CPU necessário e foi encerrado com sucesso.

8. CONCLUSÃO

8.1 – Reflexões

O projeto forneceu aos alunos envolvidos um conhecimento mais aprofundado em escalonamento de processos.

8.2 – Limitações conhecidas

Simulação Lógica Discreta Síncrona: O tempo é simulado de forma linear em um laço único sequencial (tempo++). Desse modo, não há paralelismo ou concorrência real baseada em Threads do sistema operacional anfitrião para gerenciar os dispositivos de E/S e CPU.

Apenas uma Chamada de I/O por Processo: Pela modelagem atual, cada processo gerado aleatoriamente possui suporte para disparar no máximo uma única requisição de I/O durante todo o seu ciclo de vida.

Ausência de Mecanismo de Aging (Envelhecimento): O escalonador prioriza rigorosamente a Fila Alta. Caso o sistema sofresse uma entrada infinita e ininterrupta de processos prioritários ou rápidos, os processos relegados à Fila Baixa sofreriam starvation crônico (fome de CPU), pois não há uma regra para subir a prioridade por tempo de espera.

9. BIBLIOGRAFIA

TecnoDigital (2026); Planejamento Round Robin: Definição e Exemplos; Disponível em: <https://informatecdigital.com/pt/defini%C3%A7%C3%A3o-e-exemplos-de-planejamento-round-robin/>; Acesso em 6 jun. 2026.